

**Pipeline de Observabilidad End to End con OpenTelemetry:
Instrumentación de Dos Microservicios en GCP y AWS**

Ernesto Ilich Contreras H.
Saud Mauricio Gonzalez R.

María Fernanda Ochoa Paipilla

Maestría en Inteligencia Artificial
Universidad de la Sabana

Bogotá D.C.
Agosto 2026

Resumen

Este trabajo construye un pipeline de observabilidad completo sobre dos microservicios que se llaman entre sí por HTTP y que consultan una base de datos. Los servicios emiten los tres pilares con el SDK de OpenTelemetry, un Collector los recibe y los reparte a los backends de las dos nubes, y todo queda unido por un mismo *trace_id*. El despliegue se define con Terraform y Helm, tanto para Google Kubernetes Engine como para Amazon ECS.

Lo que distingue este trabajo es que las cifras no son estimaciones tomadas de la literatura. El repositorio incluye un banco de pruebas que levanta los servicios, aplica carga y mide. Con la configuración que captura todo, la instrumentación cuesta 55,6 por ciento de latencia mediana y 26,5 por ciento de capacidad de proceso. Bajar el muestreo de cabecera al 10 por ciento reduce el volumen de spans un 88,3 por ciento y recupera parte de esa capacidad.

Palabras clave: Observabilidad, OpenTelemetry, trazas distribuidas, correlación de señales, microservicios, sobrecarga de instrumentación.

Pipeline de Observabilidad End to End con OpenTelemetry

Un sistema de microservicios reparte una sola petición entre varios procesos, y con eso rompe la forma tradicional de investigar fallas. Ya no basta con leer el registro de un servidor, porque la causa de una respuesta lenta puede estar tres saltos más allá. La observabilidad busca resolver eso con tres tipos de señales: métricas que dicen qué tan sano está el sistema, trazas que muestran el recorrido de una petición y registros que explican qué pasó en cada paso.

El problema es que tener las tres señales no significa que estén conectadas. Si las métricas viven en una herramienta, las trazas en otra y los registros en una tercera, la persona que investiga

termina comparando marcas de tiempo a mano. La conexión se logra cuando las tres comparten un identificador común, y ese identificador es el `trace_id`.

Este documento describe la construcción de ese pipeline sobre una arquitectura de dos servicios, su despliegue en dos nubes distintas y la medición del costo que la instrumentación agrega. El trabajo se organiza en cuatro fases: instrumentación con el SDK, despliegue del Collector, visualización y correlación, y análisis de sobrecarga.

Arquitectura objetivo

El flujo se mantiene simple a propósito, para que el análisis se concentre en la telemetría y no en la lógica de negocio. Una petición de compra entra por *service-a*, que consulta su base de datos para conocer el descuento del cliente, calcula el total y llama a *service-b* para reservar inventario. El segundo servicio consulta y actualiza su propia base de datos y responde. Esa petición, que dura cerca de 38 milisegundos, genera 20 spans repartidos entre los dos servicios.

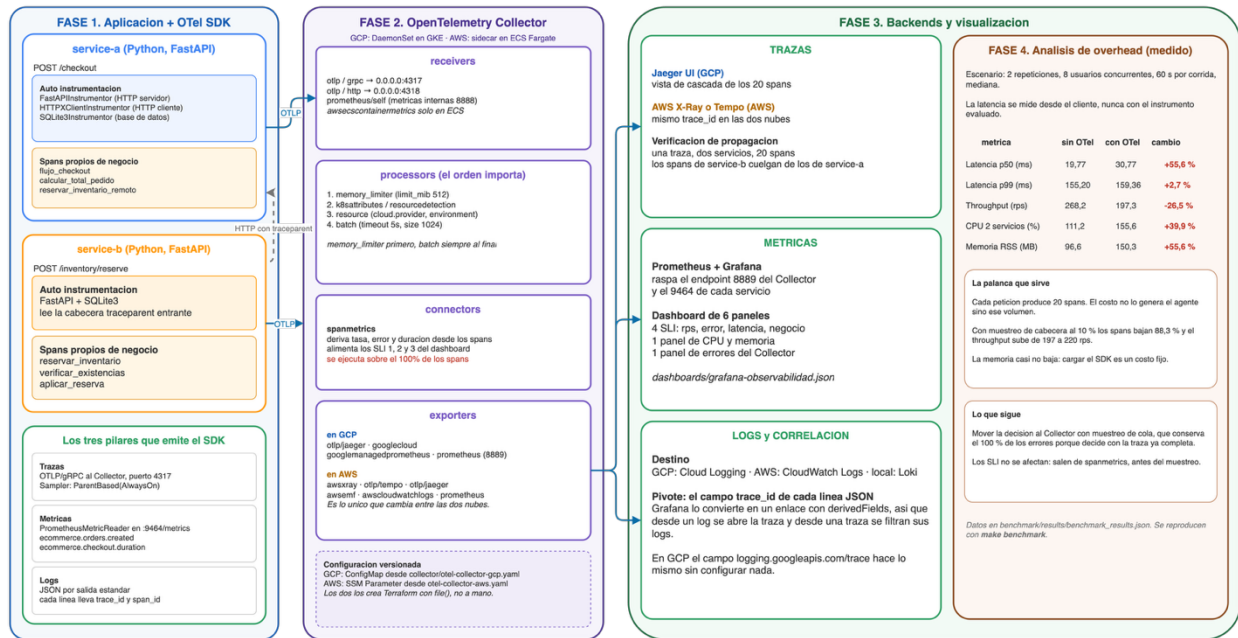
La Figura 1 presenta el recorrido completo de la telemetría, desde el proceso de la aplicación hasta los tableros.

Figura 1

Pipeline de observabilidad end to end con OpenTelemetry

Pipeline de observabilidad end to end con OpenTelemetry

service-a → service-b · GCP GKE y AWS ECS Fargate · trazas, métricas y logs unidos por trace_id



Nota. Las cuatro fases corresponden a las que plantea la actividad. Lo único que cambia entre Google Cloud y Amazon Web Services es el bloque de exporters del Collector, lo que hace que el código de los servicios sea portable. Elaboración propia.

Decisiones de diseño

Cuatro decisiones explican por qué el pipeline quedó con esta forma. Cada una se tomó descartando una alternativa que parecía más sencilla al principio.

Tabla 1

Decisiones de diseño del pipeline y alternativas descartadas

Decisión	Alternativa descartada	Razón
La aplicación solo habla con el Collector, nunca con un backend	Exportar directo a Cloud Trace y a X-Ray	Cambiar de proveedor obligaría a tocar el código de los dos servicios. Con el Collector es editar un archivo de configuración.
Un solo módulo de arranque para los dos servicios	Configurar el SDK dentro de cada aplicación	Evita que los dos servicios se desincronicen y permite apagar toda la telemetría con una variable de entorno, que es lo que exige el banco de pruebas.
El conector spanmetrics vive en el Collector	Calcular tasa, error y duración con métricas escritas a mano	El Collector ve el cien por ciento de los spans, así que los indicadores de servicio no dependen del muestreo que se aplique después.

Decisión	Alternativa descartada	Razón
Instrumentación automática como base y spans propios solo para negocio	Escribir todos los spans a mano	La instrumentación automática cubre HTTP y base de datos sin tocar el código. Lo manual se reserva para lo que ninguna librería puede conocer.

Nota. Elaboración propia. Las cuatro decisiones se implementan en el repositorio y se pueden verificar en los archivos que indica el Anexo A.

El mismo diseño en dos nubes

Los receivers y los processors del Collector son idénticos en las dos nubes. La diferencia está en dónde corre el proceso y hacia dónde exporta, como resume la Tabla 2.

Tabla 2

Diferencias de despliegue entre Google Cloud y Amazon Web Services

Aspecto	Google Cloud (GKE)	Amazon Web Services (ECS Fargate)
Forma de despliegue	DaemonSet, un Collector por nodo	Contenedor sidecar en cada task definition
Trazas	otlp/jaeger, googlecloud	awsxray, otlp/tempo, otlp/jaeger
Métricas	prometheus, googlemanagedprometheus	prometheus, awsemf
Registros	googlecloud	awscloudwatchlogs
Configuración	ConfigMap que crea Terraform	Parámetro de SSM que crea Terraform
Identidad	Workload Identity	Rol IAM de la tarea

Nota. En ECS Fargate no existe el concepto de DaemonSet, por eso el patrón equivalente es el contenedor sidecar. La aplicación le habla por localhost, que tiene el mismo costo que hablarle a un agente en el mismo nodo.

Fase 1: instrumentación con el SDK

La división del trabajo entre instrumentación automática y manual sigue una regla simple: la automática cubre lo que es igual en cualquier servicio y la manual cubre lo que solo el equipo de producto entiende.

Tabla 3

Reparto entre instrumentación automática y manual

Tipo	Qué captura	Spans que produce
Automática	FastAPIInstrumentor, el HTTP de entrada	POST /checkout, POST /inventory/reserve
Automática	HTTPXClientInstrumentor, el HTTP de salida	POST, e inyecta la cabecera traceparent
Automática	SQLite3Instrumentor, la base de datos	SELECT, INSERT y UPDATE con el atributo db.statement
Manual	El flujo de compra de service-a	flujo_checkout, calcular_total_pedido, reservar_inventario_remoto
Manual	La reserva de service-b	reservar_inventario, verificar_existencias, aplicar_reserva

Nota. La instrumentación automática se activa con variables de entorno y no toca el código de negocio. Los spans manuales agregan atributos propios bajo el prefijo ecommerce.

Un detalle práctico que costó depurar merece mención, porque no aparece en la documentación de forma evidente. La instrumentación automática de bases de datos envuelve el cursor, no la conexión. Si el código llama a `con.execute()` en lugar de `cur.execute()`, no se genera ningún span de base de datos y el problema pasa desapercibido, porque todo lo demás sigue funcionando. El repositorio usa cursores explícitos por esa razón.

Los tres pilares

Cada pilar sale por un camino distinto y se verificó de forma independiente, como muestra la Tabla 4.

Tabla 4

Emisión y verificación de los tres pilares de observabilidad

Pilar	Cómo sale del servicio	Dónde se verificó
Trazas	OTLP sobre gRPC hacia el Collector, puerto 4317	20 spans capturados en el receptor OTLP
Métricas	PrometheusMetricReader en el puerto 9464	37 series en docs/evidencia/metricas_service_a.txt
Registros	JSON por salida estándar con trace_id y span_id	5 líneas en docs/evidencia/logs_correlacionados.jsonl

Nota. Las métricas de negocio que expone el SDK son ecommerce_orders_created_total, ecommerce_checkout_duration_seconds, ecommerce_order_amount, inventory_reservations_total, inventory_reserve_duration_seconds e inventory_stock_level.

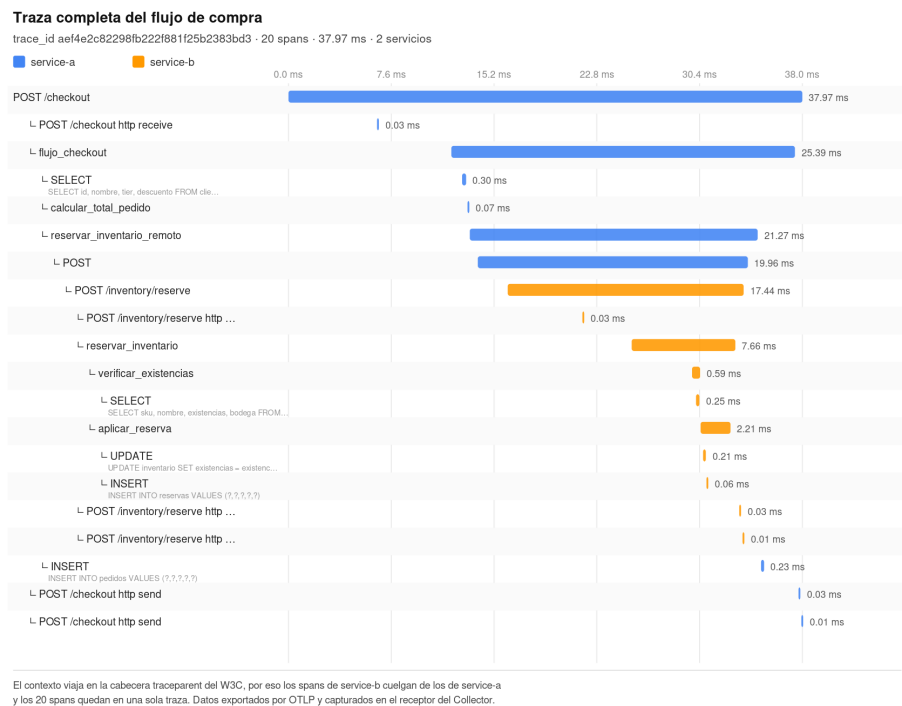
Fase 3: propagación de contexto y correlación

La propagación es la pieza que decide si el sistema tiene trazas distribuidas o dos conjuntos de trazas sueltas que nadie puede relacionar. El mecanismo es el siguiente: *service-a* crea el identificador de traza, el cliente HTTP instrumentado lo escribe en la cabecera `traceparent` definida por el Consorcio World Wide Web (2021), y *service-b* la lee y continúa la misma traza en lugar de empezar una nueva.

La Figura 2 es la prueba de que el mecanismo funciona. Se construyó desde los spans reales que llegaron al receptor OTLP durante una petición de compra.

Figura 2

Traza completa de una petición de compra capturada en el entorno de pruebas



Nota. Los spans azules pertenecen a *service-a* y los naranjas a *service-b*. Que los naranjas cuelguen de los azules es lo que demuestra que el contexto viajó entre los dos procesos. Datos exportados por OTLP y capturados en el receptor del Collector.

Tres cosas se leen en esa figura. La primera es que los 20 spans comparten un solo identificador de traza. La segunda es que la jerarquía es correcta, porque el span de servidor de *service-b* es hijo del span de cliente HTTP de *service-a*. La tercera es dónde se va el tiempo: la llamada a *service-b* concentra 21,27 de los 37,97 milisegundos, es decir el 56 por ciento.

Las piezas que hacen posible la correlación

Que las tres señales existan no significa que estén conectadas. La conexión depende de tres piezas concretas, y si falta cualquiera de ellas la investigación vuelve a ser una comparación manual de marcas de tiempo.

Tabla 5

Piezas que habilitan la correlación entre los tres pilares

Pieza	Qué hace	Dónde está definida
LoggingInstrumentor con inject_trace_context	Agrega el identificador de traza a cada registro, que el formateador escribe como el campo trace_id	services/common/telemetry.py
derivedFields de Grafana	Convierte el campo trace_id de un registro en un enlace que abre la traza correspondiente	dashboards/grafana-datasources.yml
Atributos de recurso compartidos	service.name y deployment.environment.name identifican el mismo servicio en las tres señales	Resource del SDK y processor resource del Collector

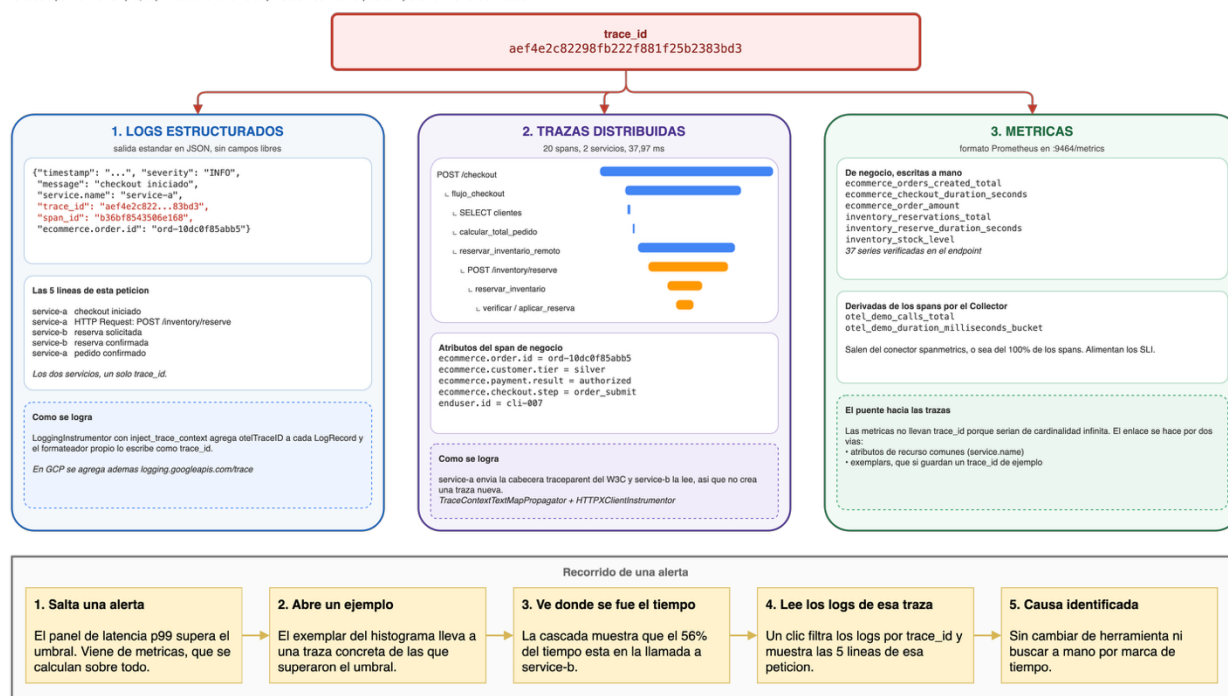
Nota. En Google Cloud existe una cuarta pieza que ahorra configuración. Si el registro incluye el campo logging.googleapis.com/trace, Cloud Logging enlaza con Cloud Trace sin configuración adicional. El formateador lo agrega cuando existe la variable GCP_PROJECT_ID.

Figura 3

Correlación entre los tres pilares usando el identificador de traza como pivote

Correlacion entre los tres pilares usando trace_id como pivote

Una sola peticion de compra, capturada en el entorno de pruebas. Los tres bloques comparten el mismo identificador.



Nota. Los tres bloques corresponden a una misma peticion capturada en el entorno de pruebas. La franja inferior muestra el recorrido que sigue una persona desde una alerta hasta la causa, sin cambiar de herramienta. Elaboración propia.

Procedimiento de verificación

La correlación se comprueba en cuatro pasos que cualquier persona puede repetir:

```
# 1. genere una peticion y guarde el identificador
curl -s -X POST localhost:8001/checkout \
  -H 'Content-Type: application/json' \
  -d '{"cliente_id":"cli-001","sku":"SKU-1001","cantidad":2}' | jq -r .trace_id

# 2. en Jaeger debe aparecer UNA traza con 20 spans y dos servicios

# 3. en Grafana Explore, con Loki:
{container=~"service-.*"} | json | trace_id = "<el identificador>"
#   deben salir 5 lineas: 3 de service-a y 2 de service-b

# 4. el enlace TraceID de cualquier linea abre esa misma traza
```

El resultado de esa comprobación está guardado en la carpeta de evidencia del repositorio.

Las cinco líneas son: Checkout iniciado, la llamada HTTP saliente y pedido confirmado en *service-a*, más reserva solicitada y reserva confirmada en *service-b*.

Fase 4: Análisis de Sobrecarga

Método de medición

El mismo binario corre en los dos escenarios. La variable `OTEL_ENABLED` apaga el SDK completo, de modo que el código de negocio es idéntico y la única diferencia entre las corridas es la telemetría. El protocolo fue el siguiente:

- Dos escenarios, sin instrumentación y con instrumentación completa.
- Dos repeticiones de 60 segundos por escenario, y se reporta la mediana.
- Los primeros 12 segundos de cada corrida se descartan por calentamiento.
- La latencia se mide desde el cliente. Medirla dentro del servicio instrumentado sería evaluar el instrumento consigo mismo.
- El uso de procesador y de memoria se lee de los procesos de aplicación. El Collector se mide aparte, porque es infraestructura compartida y no costo de la aplicación.
- Un tercer escenario agrega muestreo de cabecera al 10 por ciento, para medir el efecto de la principal palanca de control.

El entorno fue un contenedor con 4 procesadores virtuales, 3.904 megabytes de memoria y Python 3.10.12. El generador de carga corre en la misma máquina, por lo que las latencias absolutas incluyen la competencia por procesador entre el generador y los servicios. La comparación entre escenarios sigue siendo válida porque las dos corridas sufren esa misma condición.

Resultados

Tabla 6

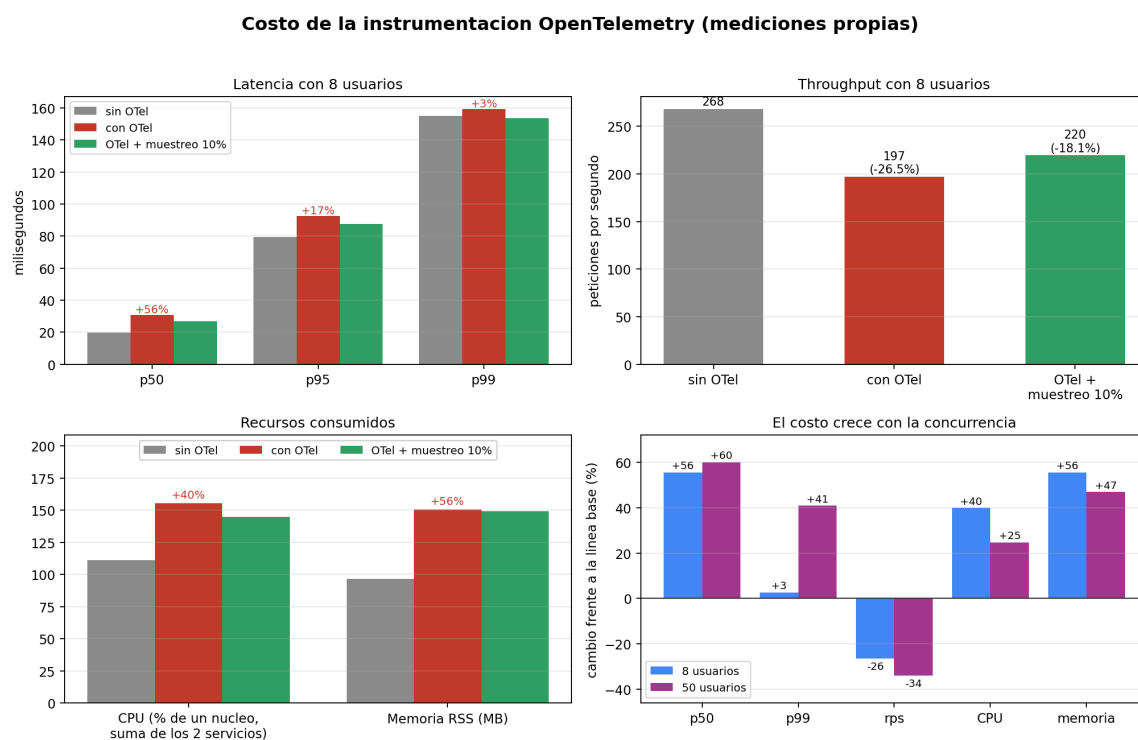
Costo de la instrumentación con ocho usuarios concurrentes

Métrica	Sin OTel	Con OTel	Cambio	Muestreo 10%
Latencia p50 (ms)	19,77	30,77	+55,6 %	26,92
Latencia p95 (ms)	79,41	92,57	+16,6 %	87,57
Latencia p99 (ms)	155,2	159,36	+2,7 %	153,65
Peticiones por segundo	268,2	197,25	-26,5 %	219,6
Procesador (%)	111,15	155,55	+39,9 %	144,7
Memoria RSS (MB)	96,6	150,3	+55,6 %	149,3

Nota. Mediana de dos repeticiones de 60 segundos. El porcentaje de procesador es la suma de los dos servicios sobre un núcleo. El Collector consumió aparte 4,2 por ciento de un núcleo y 285,6 megabytes, procesando 235.928 spans en 60 segundos.

Figura 4

Comparación del costo de la instrumentación en los distintos escenarios



Nota. El panel inferior derecho compara el costo relativo con ocho y con cincuenta usuarios concurrentes. Elaboración propia con los datos de benchmark/results/benchmark_results.json.

Interpretación de los resultados

La latencia mediana sube mucho más que la del percentil 99

La latencia mediana crece 55,6 por ciento mientras que el percentil 99 solo crece 2,7 por ciento. La razón es que el costo de la instrumentación es casi constante por petición, alrededor de 11 milisegundos. Sobre una petición rápida ese costo pesa mucho y sobre una lenta se diluye entre el tiempo de espera. Conviene tenerlo presente al diseñar tableros, porque un panel que solo vigile el percentil 99 va a subestimar el impacto real.

La capacidad de proceso cae más de lo que sugiere la latencia

Se pierden 71 peticiones por segundo, es decir 26,5 por ciento. Con el uso de procesador subiendo 39,9 por ciento, la explicación es directa: La instrumentación compite por el mismo recurso que atiende peticiones. Este dato importa más que la latencia para dimensionar la infraestructura, porque se traduce en más réplicas.

La memoria es un costo fijo, no proporcional

La memoria sube 55,6 por ciento y casi no baja al activar el muestreo: 149,3 megabytes frente a 150,3. Es el costo de cargar el SDK y las librerías de instrumentación, y se paga aunque no se exporte un solo span. Reducir el volumen de telemetría no ayuda en esta dimensión.

El costo crece con la concurrencia

Con cincuenta usuarios la latencia mediana sube 60,1 por ciento y la capacidad cae 34,1 por ciento, frente a 55,6 y 26,5 por ciento con ocho usuarios. Cuando la máquina ya está cerca de su límite, cualquier trabajo adicional se paga con creces. Es un argumento a favor de medir la sobrecarga bajo la carga real de producción y no bajo una carga cómoda.

La palanca que sí funciona

Cada petición produce 20 spans, que es mucho para un flujo de dos servicios. El costo no viene del agente sino de ese volumen. Bajar el muestreo de cabecera al 10 por ciento cambia el balance de forma notoria, como muestra la Tabla 7.

Tabla 7

Efecto del muestreo de cabecera al 10 por ciento

Indicador	Sin muestreo	Muestreo al 10%	Cambio
Spans exportados en 60 s	235.928	27.520	-88,3 %
Peticiones por segundo	197,25	219,6	+11,3 %
Latencia p50 (ms)	30,77	26,92	-12,5 %
Procesador (%)	155,55	144,7	-7,0 %
Memoria RSS (MB)	150,3	149,3	-0,7 %

Nota. El muestreo de cabecera se configura con la variable OTEL_SAMPLE_RATIO y no requiere cambios en el código de negocio.

El muestreo de cabecera tiene un problema conocido: Decide antes de saber si la petición falló, así que se pierden errores poco frecuentes. Para un entorno de producción la recomendación es mover la decisión al Collector con muestreo de cola, que conserva el cien por ciento de los errores porque decide cuando la traza ya está completa. Los indicadores de nivel de servicio no se ven afectados en ninguno de los dos casos, porque salen del conector spanmetrics, que se ejecuta antes del muestreo.

Conclusiones

El pipeline cumple lo que se propuso. Una petición produce una sola traza que atraviesa los dos servicios, cinco líneas de registro que llevan ese mismo identificador y un conjunto de métricas que se calculan sobre el total de peticiones. Desde una alerta se llega a la causa sin cambiar de herramienta ni comparar marcas de tiempo a mano.

Tres lecciones quedan para llevar a un entorno de producción.

Medir antes de opinar. La intuición sugería que la sobrecarga sería de un solo dígito. Resultó ser de 55,6 por ciento en latencia mediana y de 26,5 por ciento en capacidad. Sin un banco de pruebas ese dato no habría aparecido hasta que apareciera en producción.

El volumen es la variable de control. Veinte spans por petición es demasiado para un flujo de dos servicios. La respuesta correcta no es quitar la instrumentación sino revisar qué spans aportan a un diagnóstico y ajustar el muestreo.

El orden dentro del Collector protege los indicadores. Colocar el conector spanmetrics antes de cualquier muestreo permite reducir el volumen de trazas sin mover los indicadores de nivel de servicio, que se siguen calculando sobre el cien por ciento de las peticiones.

Quedan tres tareas pendientes. La primera es repetir la medición en las nubes, con el generador de carga fuera del anfitrión, para separar el costo de la instrumentación del de la máquina. La segunda es sustituir el muestreo de cabecera por muestreo de cola en el Collector, con políticas que conserven todos los errores. La tercera es revisar el mapa de spans y retirar los que no aportan a un diagnóstico, empezando por los spans internos de recepción y envío del protocolo ASGI.

Referencias

- Grafana Labs. (2026). *k6 load testing documentation*. <https://k6.io/docs/>
- Grafana Labs. (2026). *Trace integration: Linking traces, logs and metrics*.
<https://grafana.com/docs/grafana/latest/explore/trace-integration/>
- Jaeger. (2026). *Architecture documentation*. <https://www.jaegertracing.io/docs/architecture/>
- OpenTelemetry. (2026). *Collector documentation*. <https://opentelemetry.io/docs/collector/>
- OpenTelemetry. (2026). *Python SDK documentation*. <https://opentelemetry-python.readthedocs.io/>
- World Wide Web Consortium. (2021). *Trace context (W3C Recommendation)*.
<https://www.w3.org/TR/trace-context/>

Anexo A

Ubicación de los entregables dentro del repositorio

Tabla A1

*Correspondencia entre los entregables y los archivos del repositorio GitHub **otel-observability-pipeline***

Entregable	Ubicación
Código de instrumentación con el SDK	services/common/telemetry.py, services/service-a, services/service-b
Configuración del OTel Collector	collector/otel-collector-gcp.yaml, -aws.yaml y -local.yaml
Manifiestos de infraestructura como código	iac/terraform/gcp, iac/terraform/aws, iac/helm/otel-collector, iac/k8s
Capturas de Jaeger con trazas completas	docs/figuras/fig2_traza_completa.png y la guía docs/CAPTURAS.md
Tableros de Grafana	dashboards/grafana-observabilidad.json, seis paneles y el de correlación
Reporte técnico	docs/Informe_Actividad_4_SMGR_EICH_20260816.pdf
Prueba de carga	benchmark/load_test.js para k6 y benchmark/run_benchmark.py
Datos crudos de la medición	benchmark/results/benchmark_results.json y corridas.jsonl
Evidencia de la correlación	docs/evidencia con la traza, los registros y las métricas capturados

Nota. El repositorio se ejecuta completo en local con el archivo docker-compose.yaml incluido en la raíz.